

Pi-Agent 与 DeepSeek Harness 架构深度洞察报告

文档定位：从架构范式、内核设计、运行时机制、扩展模型、状态管理、记忆系统集成等维度，对 Pi-Agent (pi-mono) 和 DeepSeek Harness (dsh) 两个 Agent 框架进行源码级深度拆解和对比分析。不是功能罗列，而是架构洞察——回答“为什么这样设计”和“代价是什么”。

>

撰写日期：2026年9月8日

信息来源：GitHub 仓库源码与架构文档 (docs/architecture.md)、DeepWiki 代码分析、社区深度技术评论 (Dwarves Memo、rustman、dsh-in-depth、DSHKit)、Cordis 学术论文 (arXiv:2608.25512)、Mem0 官方插件源码、Pi 官方 README 与 mudrli/pi-mono-docs 架构概览
文档版本：v1.0

目录

1. 引言：两个框架的时代背景
2. Pi-Agent 架构深度拆解
3. DeepSeek Harness 架构深度拆解
4. 核心机制对比：六个维度的正面交锋
5. 记忆系统架构对比与启示
6. 架构成熟度与风险评估
7. 对 Agent 框架设计范式的影响
8. 总结与选型建议

1. 引言：两个框架的时代背景

1.1 Agent Harness 概念的兴起

2026年，AI Agent 领域出现了一个清晰的共识：Agent = Model + Harness。

模型负责生成下一个 token、推理、决策；Harness

负责其余一切——工作空间、工具注册表、权限管理、会话状态、记忆持久化、Agent 循环驱动。Claude Code、Codex CLI 是这一范式的先行者，但它们是密封的产品——固定外壳，通过 MCP/API 扩展。

Pi-Agent 和 DeepSeek Harness 代表了新一代的尝试：开源、可修改的 Harness。但两者对“可修改”的理解截然不同，这种差异是本文的核心议题。

1.2 两个框架的速写

包	npm 名	职责
pi-ai	@earendil-works/pi-ai	统一多 Provider LLM API，支持 31 个 Provider ID (OpenAI、Anthropic、Google、Mistral、Bedrock、Fireworks 等)，通过子路径导出延迟加载，避免打包未用 SDK
pi-tui	@earendil-works/pi-tui	终端 UI 框架，差分渲染、组件系统 (Text、Editor、Markdown、SelectList、Image、Overlays)、内联图片、自动补全
pi-agent-core	@earendil-works/pi-agent-core	通用 Agent 运行时——状态管理、工具执行循环、事件流、上下文转换管道
pi-coding-agent	@earendil-works/pi-coding-agent	面向用户的 CLI 应用——Session 管理、扩展系统、技能、提示模板、主题
pi-web-ui	@earendil-works/pi-web-ui	Web 组件 (mini-lit + Tailwind CSS v4)，带附件、IndexedDB 存储、CORS 代理

架构洞察：三层分离意味着每层可独立使用。pi-ai 可单独用于任何 LLM 集成；pi-agent-core 可驱动与编码无关的 Agent；pi-coding-agent 只是消费者。这种分层强制了关注点分离——对基础包的修改需要谨慎，因为会影响上层所有消费者。

2.2 Agent 循环：418 行的状态机

Pi 的核心 Agent 循环位于 packages/agent/src/agent-loop.ts，是一个约 418 行的函数式状态机。它的核心逻辑异常简洁：

```
1. steeringQueue followUpQueue
2. AgentContext + +
3. pi-ai API
4. →
  a. pre/post
  b.
  c. 2
5. → → 1
6. → Compaction
```

关键设计决策：

决策	实现	理由
循环不可替换	固定代码，非配置驱动	简单、可预测、易于调试
事件钩子可干预	20+ 生命周期事件 (tool_call、message_update、turn_start 等)	给扩展干预能力，不牺牲核心稳定性
工具执行可并行	parallel 或 sequential 模式	性能优化
Compaction 内置	自动+手动，支持分支摘要	长对话必需

架构洞察：Pi 的 Agent 循环是"最小正确答案"。418 行代码包含了完整的工具执行、状态管理、事件流和上下文压缩——这是对"Agent 框架需要多少代码？"这个问题的回答：不多，如果你知道什么该做、什么不该做。

2.3 AgentHarness：高层编排层

在 Agent 循环之上，Pi 提供了 AgentHarness——一个高层编排器：

职责	实现
Session 持久化	JSONL 树结构，每个消息是树节点
资源管理	技能和提示模板的按需加载
操作锁	防止并发操作冲突
分支导航	/tree 命令在会话树中导航

Session 格式——JSONL 树：

```
session.jsonl
■■■ user_message_1 (root)
 ■■■■ assistant_message_1
  ■ ■■■■ tool_call_1
   ■ ■ ■■■■ tool_result_1
    ■ ■ ■■■■ assistant_message_2
     ■ ■ ■■■■ user_followup_2 ← ■■■■■■
      ■ ■ ■■■■ branch_A_message
       ■ ■ ■■■■ branch_B_message ← ■■■■■■
        ■ ■■■■ user_message_2 (steering)
```

架构洞察：JSONL 树结构让 Pi 支持原地分支——不需要复制会话，只需切换叶子节点。整个历史保留，可以随时回到任何分支点。这比传统的线性会话日志更强大，但读取和压缩逻辑也更复杂。

2.4 扩展系统：四种方式，一个 API

Pi 有四种扩展方式，按复杂度递增：

2.4.1 Prompt Templates（提示模板）

Markdown 文件，放在 `~/.pi/agent/prompts/` 或 `.pi/prompts/`。用 `/name` 展开。最简单的扩展方式——不需要写代码。

2.4.2 Skills（技能）

YAML 前置 Markdown 文件，遵循 Agent Skills 标准 (agentskills.io)。渐进式披露——只有描述 (description) 常驻上下文，完整内容在触发条件匹配时按需加载。

```
---
name: deploy
description: "Use when deploying to production. Handles rollback safety."
trigger: "deploy"
---
# Deploy Skill
1. Run `npm run build`
2. Run `docker compose up -d`
3. Verify with `curl localhost:8080/health`
...
```

2.4.3 Themes（主题）

TUI 外观热重载，无需重启。

2.4.4 Extensions (扩展)

TypeScript 模块，通过 `jiti` (运行时 TypeScript 加载器) 加载，无需预编译。这是 Pi 最强大的扩展方式：

```
export default function (pi: ExtensionAPI) {
  // ■■■■
  pi.registerTool({
    name: "deploy",
    description: "Deploy to production",
    schema: Type.Object({ service: Type.String() }),
    handler: async (args) => { /* ... */ }
  });

  // ■■■■
  pi.registerCommand("stats", {
    description: "Show session stats",
    handler: async () => { /* ... */ }
  });

  // ■■■■——20+ ■■■■■■
  pi.on("tool_call", async (event, ctx) => {
    // ■■■■■■/■■■■■
  });

  pi.on("turn_start", async (event) => { /* ... */ });
  pi.on("message_update", async (event) => { /* ... */ });

  // ■■■■■■■■UI widget■CLI ■■■■■■■■
  pi.registerKeybinding("Ctrl+D", () => { /* ... */ });
}
```

扩展发现机制 (三个位置)：

位置	范围	示例
~/.pi/agent/extensions/	全局	个人的常用扩展
.pi/extensions/	项目级	项目特定的扩展
Pi Packages (npm/git)	可分享	pi install npm:@mem0/pi-agent-plugin

架构洞察：Pi 的扩展 API 有 20+ 生命周期钩子，覆盖了 Agent 运行的每个阶段。但关键限制是——扩展只能加/覆盖工具和钩子，不能替换核心循环。如果你需要修改 Agent 的基本行为模式 (比如改成多 Agent 协调)，你需要 fork 代码或用扩展在外部实现 (如用 `tmux` 起 Pi 实例)。

2.5 pi-ai : 31 个 Provider 的统一抽象

pi-ai 包是 Pi 的 LLM 抽象层，支持 31 个已发布的 Provider ID。关键设计：

特性	实现
延迟加载	Provider 通过子路径导出 (如 @earendil-works/pi-ai/anthropic) , 避免打包未用 SDK
工具定义	TypeBox 1.x schema (而非 JSON Schema)
Token 追踪	自动 token 和成本追踪
跨 Provider 切换	Session 可包含来自多个 Provider 的消息, thinking traces 在跨 Provider 时归一化为文本
模型发现	自动发现可用模型

架构洞察：Pi 原生支持 DeepSeek 模型，DeepSeek 官方 API 文档有 Pi 集成教程。/model 命令或 Ctrl+L 可热切换模型。这使得 Pi 成为使用 DeepSeek 模型的自然选择。

2.6 四种运行模式

模式	命令	用途
Interactive	pi	完整 TUI 体验，差分渲染
Print	pi -p "query"	脚本化，输出后退出
JSON	pi --mode json	事件流 JSON Lines，供程序消费
RPC	pi --mode rpc	stdin/stdout JSONL 协议，非 Node.js 集成
SDK	import { createAgentSession }	嵌入 Node.js 应用 (OpenClaw 就是这样做的)

2.7 Pi 的"故意不做"清单

这是理解 Pi 架构的关键——以下功能 Pi 故意不内置：

功能	Pi 的立场	理由
No MCP	不内置 MCP 协议	写 CLI 工具 + README 就够了，或用扩展加
No sub-agents	不内置子 Agent	用 tmux 起 Pi 实例，或用扩展
No permission popups	不内置权限弹窗	跑容器里，或用扩展建确认流程
No plan mode	不内置计划模式	写到文件里，或用扩展建
No built-in to-dos	不内置 TODO	会混淆模型，用 TODO.md 文件
No background bash	不内置后台 bash	用 tmux，完全可观察

架构洞察：Pi 的哲学不是"功能不够"，而是"故意不做"。Mario Zechner 的主张是：其他 Agent 框架内置的这些功能，要么会混淆模型 (TODO 列表)，要么有更好的替代方案 (tmux 代替 background bash)，要么应该让用户自己选 (权限系统)。核心越小，扩展空间越大。CONTRIBUTING.md 明确写道："如果你的功能不属于核心，它应该是扩展。膨胀核心的 PR 会被拒绝。"

含义：任何能力——包括 Agent 循环本身——都可以被替换。这不是 API 契约，这是架构约束。core/agent 拥有接口定义，core/agent-loop 只是默认驱动器实现——你可以替换 agent-loop 而不动 agent 接口。

不变量 2：模型可见 = 已记录

"任何到达模型请求的内容都必须能从日志重建。运行时不变量会断言这一点。"

这是 dsh 最核心的设计——会话日志是唯一真相源。三个机制保证：

1. 结构性保证：模型请求只能从日志推导（`deriveMessages()`），没有第二条代码路径
2. 写入前校验：每个事件在被追加到日志前都经过验证
3. 运行时断言：断言插件逐字节比对外发请求与从日志重建的请求，不匹配则中止

含义：没有侧信道可以让信息进入模型上下文。如果你想注入记忆，你必须通过会话事件系统完成，确保可追溯、可回放、可审计。

不变量 3：后层覆盖前层

"补丁按行替换整个配置值，不做深度合并。后层胜出。"

层叠顺序：

- ```
1. ■ bundle ■ cordis.patch.yml■■ profile ■■■■■■
2. Profile ■■■ cordis.patch.yml
3. Home ■ cordis.patch.yml
4. ■■■ --patch ■■
```

要改一个字段必须重述该行所有键——不做深度合并。这避免了“不知道哪个层的哪个键生效”的调试噩梦。

不变量 4：一个 Bundle 永远是起始位置

"Bundle 插入的任何内容都被上层保留可补丁性。Bundle 是起始位置，永远不是密封的盒子。"

含义：你安装的任何插件包都不能“锁定”自己的行为。上层的 Patch 总是可以覆盖它插入的配置行。

### 3.4 启动过程：从空列表到运行时

```

■■ dsh --profile web
■
▼
Step 1: ■■ Profile ■■
~/.dsh/profiles/web/
· bundle [■■■■■■■■][dsh-base, dsh-web-app]
· ■■■■ npm [■■■■■■■■]
· cordis.patch.yml■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■[]
← ■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■
■
▼
Step 2: ■■■■ Patch ■
Layer 1: dsh-base ■ cordis.patch.yml
→ ■■■■■■■■■■■■■■■■■■■■

```

### 3.5 控制脊柱与能力缝接 (Seam)

### 3.5.1 六个核心服务（控制脊柱）

— 10 —

| 服务       | ctx 键   | 职责             |
|----------|---------|----------------|
| 模型适配器注册表 | ctx.llm | 消息和流词汇 + 适配器接缝 |

### 3.5.2 能力缝接 (Seam)

Seam ( 缝接 ) 是 dsh 中可替换能力的抽象接口，有三个角色：

| 角色                 | 职责   | 类比          |
|--------------------|------|-------------|
| Service Definition | 声明接口 | 接口/Protocol |
| Service Provider   | 实现接口 | 具体类         |
| Consumer           | 使用接口 | 调用方         |

"一个包可以合并角色，但只有一个角色不是 Seam；添加一个能力意味着设计全部三个。"

核心 Seam 列表：

| Seam    | ctx 键                  | 作用                                           |
|---------|------------------------|----------------------------------------------|
| 文件系统    | ctx.fs                 | 文件系统访问和策略                                    |
| 子进程     | ctx.subprocess         | 进程生成                                         |
| Shell   | ctx.shell              | Shell 执行后端                                   |
| 终端      | ctx.terminals          | 持久终端执行                                       |
| 沙箱      | ctx.sandbox            | 进程限制                                         |
| 命令      | ctx.commands           | 人类命令分发                                       |
| 后台任务    | ctx.jobs               | 后台工作注册                                       |
| Webhook | ctx.webhookRuntime     | 认证投递分发和会话创建                                  |
| 会话标题    | ctx.sessionTitle       | 会话标题生成                                       |
| 目标管理    | ctx.goals              | 同会话目标管理                                      |
| 会话投影    | ctx.sessionProjections | 增量折叠已提交事件                                    |
| 子 Agent | (subagent)             | 可为进程内子 Agent、ACP 对等体、Codex 进程、Claude Code 进程 |

### 3.5.3 Seam 的连锁效应

"文件系统和子进程提供者共享一个执行世界，所以将它们指向远程沙箱会将 Bash、PTY 和 LSP 一起移动，而不需要任何提供者 fork。"

这是 Seam 设计的威力——一个提供者替换改变整个产品。你不需要为远程沙箱分别 fork Bash 工具、PTY 工具和 LSP 工具；你只需要替换文件系统和子进程提供者，所有依赖它们的工具自动跟随。

子 Agent Seam 更进一步：一个"子 Agent"可以是进程内子 Agent、ACP 对等体、Codex 进程、或 Claude Code 进程，全部在一个接口后面。dsh 被设计为可以编排其他 Harness——这使它定位为"元 Harness" ( meta-harness )。

## 3.6 Turn 流：Agent 循环的精确规范

dsh 架构文档精确定义了 Agent 循环的事件流：

### 3.6.1 基本概念

| 概念          | 定义                              |
|-------------|---------------------------------|
| Step ( 步骤 ) | 一次模型请求 + 它调用的工具                 |
| Turn ( 轮次 ) | 零个或多个步骤：在第一个输入被认领前打开，在不欠任何工作时关闭 |

### 3.6.2 完整事件流

[illegible]

### 3.6.3 事件分类

| 类型     | 事件                                                                    | 特征                  |
|--------|-----------------------------------------------------------------------|---------------------|
| 持久会话事件 | turn/*、step/*、user/message、assistant/message、assistant/attempt、tool/* | 写入只追加日志，跨重载存活       |
| 实时扩展点  | agent/*、tools/*、llm/stream                                            | 进程内事件，用于观察/拦截进行中的工作 |
| 瀑布型    | agent/pre-step、agent/request、llm/stream、tools/*                       | 监听器必须调用 next() 委托   |
| 串行型    | agent/turn-stopping                                                   | 无 next()，串行执行       |

### 3.6.4 注入机制

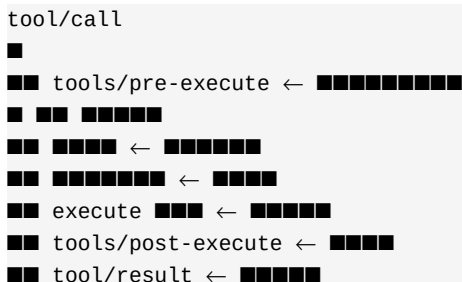
"输入通过一个收件箱到达驱动器。一些消息立即唤醒它；注入的上下文在收件箱中等待，直到另一条消息到来。"

agent/pre-step 决定模型看到什么——监听器可以重写认领的消息、拒绝消息。agent.inject() 是添加模型可见上下文的唯一正规路径——它落在下一个被准入的请求中。

架构洞察：dsh 的事件系统有四种分发模式（瀑布型、串行型、持久型、瞬时型），比 Pi 的简单事件钩子复杂得多。这种复杂度是“一切皆可替换”的代价——每个拦截点都必须是类型化的、有序的、可委托的。

### 3.7 工具执行管道

dsh 的工具调用经过一个受保护的管道：



这些拦截点与 Claude Code 的 hook 配置对应，但 dsh 使用类型化的进程内瀑布事件，而非 shell 子进程 + exit code。代价：更强大，但失去了“写个五分钟 bash 脚本”的可及性。

### 3.8 Host/Client 分离

dsh 将同一源码树构建为两个面：

| 面      | 环境      | 职责                       |
|--------|---------|--------------------------|
| Host   | Node 进程 | 运行 Web 服务器、文件系统访问、子进程、沙箱 |
| Client | 浏览器     | 运行 Web UI、编辑器、终端前端       |

通过 tsconfig.host.json 和 tsconfig.client.json 切换 ( --env.DSH\_BUILD\_FACE )。

### 3.9 Profile 与 Bundle

| 概念      | 说明                                         |
|---------|--------------------------------------------|
| Bundle  | 声明了 dsh.bundle 的 npm 包，携带 cordis.patch.yml |
| Profile | ~/.dsh/profiles/ 下的命名组装体，描述哪些 Bundle 叠加    |
| Patch   | YAML 配置行，按 id 替换整行                         |

四个内置 Profile：

| Profile  | 叠加的 Bundle              | 产物                         |
|----------|-------------------------|----------------------------|
| web      | dsh-base + dsh-web-app  | Web 工作台 ( 127.0.0.1:3080 ) |
| headless | dsh-base + dsh-headless | 无头一次性运行器 ( 无服务器、无浏览器 )     |

| Profile     | 叠加的 Bundle             | 产物                    |
|-------------|------------------------|-----------------------|
| sdk         | dsh-base + dsh-sdk-app | SDK JSON-RPC 服务器      |
| sdk-minimal | dsh-sdk-minimal ( 独立 ) | 最小 SDK 树，不应用 dsh-base |

web 和 headless 的唯一区别是堆叠了 dsh-web-app 还是 dsh-headless——同一个内核、同一个 dsh-base，不同的 UI bundle。

3.10 凭据引用模式

dsh 的凭据管理有一个值得注意的设计：

```
llm-pi-ai ■■■■
apiKeyEnv: DSH_DEEPSEEK_API_KEY # ■■■■■■■■■■■■
```

apiKeyEnv 命名一个凭据，在每次请求时解析——没有密钥进入配置文件。密钥存储在 \$DSH\_HOME/.credentials.yaml 或系统 Keychain 中，配置只持有指针。

架构洞察：两个独立到达同一结论的代码库（dsh 和 Pi 生态中的 DirectorOS）都选择了"凭据是引用，不是值"的模式。这是该设计的默认正确性信号。

3.11 Claude Code 互操作

dsh 有一个明确的策略：让 Claude Code 用户无痛迁移。

| 现有资产                  | dsh 支持                                                 | 机制                                              |
|-----------------------|--------------------------------------------------------|-------------------------------------------------|
| AGENTS.md / CLAUDE.md | <input type="checkbox"/> 原生                            | 从 root 到 cwd 遍历，去重相同内容，注入为持久上下文                 |
| Claude Code hooks     | <input type="checkbox"/> <input type="checkbox"/> 部分桥接 | hooks-claude-code 运行 shell 命令 hook 子集           |
| Skills                | <input type="checkbox"/> <input type="checkbox"/> 概念移植 | ctx.skills 文件系统发现                               |
| MCP servers           | <input type="checkbox"/>                               | 内置 MCP 客户端                                      |
| Claude Code 本身        | <input type="checkbox"/> 作为子 Agent                     | subagent-claude-code 委派一个 turn 给 Claude Code 进程 |
| 模型                    | DeepSeek 一等公民 + OpenAI 兼容                              | llm-deepseek + llm-pi-ai                        |

架构洞察：dsh 能将 Claude Code 作为子 Agent 驱动，这意味着它定位自己为"元 Harness"——编排其他 Harness。这是 Pi 做不到的（Pi 不内置子 Agent）。

4. 核心机制对比：六个维度的正面交锋

4.1 Agent 循环：固定 vs 可替换

| 维度   | Pi                          | dsh                     |
|------|-----------------------------|-------------------------|
| 循环定义 | 固定核心代码（agent-loop.ts，~418行） | Turn 流插件（ctx.agentLoop） |

| 维度    | Pi                                                 | dsh                                           |
|-------|----------------------------------------------------|-----------------------------------------------|
| 可替换性  | <input type="checkbox"/> 固定，不可从配置替换                | <input type="checkbox"/> 完全可替换，本身是插件          |
| 事件钩子  | 20+ 生命周期事件                                         | 4 种分发模式（瀑布/串行/持久/瞬时）                          |
| 并行执行  | <input type="checkbox"/> 工具可并行，循环不可                | <input type="checkbox"/> （Cordis 可做但不原生支持图并行） |
| 人机协同  | <input type="checkbox"/> 需扩展                       | <input type="checkbox"/> 审批插件 + 工具管道          |
| 上下文压缩 | <input type="checkbox"/> 内置 Compaction（自动+手动+分支摘要） | 插件实现（监听 pre-step 瀑布）                          |
| 循环可视化 | <input type="checkbox"/>                           | <input type="checkbox"/>                      |

深度分析：Pi 的循环虽不可替换，但有 20+ 事件钩子可以干预。dsh 的循环可替换，但事件系统更复杂（四种分发模式）。这是"简单+可预测"与"复杂+可组合"之间的经典权衡。

## 4.2 状态管理：JSONL 树 vs 只追加事件流

| 维度   | Pi                                    | dsh                               |
|------|---------------------------------------|-----------------------------------|
| 状态容器 | JSONL Session 文件（树结构）                 | 事件流日志（SessionEvent 流）             |
| 持久化  | 文件系统                                  | 会话日志插件                            |
| 不变量  | 无正式不变量定义                              | 模型可见 = 已记录（运行时断言）                 |
| 分支能力 | <input type="checkbox"/> 原地树结构（/tree） | <input type="checkbox"/> 从日志 fork |
| 时间旅行 | 会话分支                                  | 日志 fork + 检查点回溯                   |
| 可复现性 | Session 文件可重放                         | 逐字节可复现（运行时断言保证）                   |
| 审计能力 | 中等（Session 可查看）                       | 强（模型请求可逐字节验证）                     |

深度分析：dsh 的"模型可见 = 已记录"是一个架构级的保证——不是约定，是运行时断言。Pi 没有等价的不变量——扩展理论上可以直接修改模型上下文（虽然不推荐）。dsh 的设计更严格，但代价是每个模型可见的输入都必须声明一个持久事件类型。

## 4.3 扩展模型：TypeScript 扩展 vs Cordis 插件

| 维度    | Pi Extensions          | dsh Cordis Plugins              |
|-------|------------------------|---------------------------------|
| 扩展单位  | TypeScript 模块          | Cordis 插件                       |
| 加载方式  | jiti 运行时 TypeScript 加载 | Cordis 挂载（从配置树）                 |
| 能替换什么 | 只能加/覆盖工具和钩子            | 可以替换一切，包括 Agent 循环              |
| 卸载行为  | 扩展移除，可能留残余状态           | 完全撤销——所有副作用自动回退                 |
| 依赖管理  | npm 包，手动管              | Cordis 自动追踪，兄弟变化时重新解析           |
| 配置方式  | package.json 的 pi 字段   | dsh.bundle + cordis.patch.yml   |
| 热重载   | /reload 命令             | 某些 Profile 支持 live patch reload |

| 维度     | Pi Extensions     | dsh Cordis Plugins |
|--------|-------------------|--------------------|
| UI 可替换 | 扩展可替换编辑器、加 widget | UI 本身是 bundle，整个换  |
| 类比     | 浏览器扩展             | 操作系统驱动             |

深度分析：Pi 的扩展模型更简单、更可达——写个 TypeScript 模块就行。dsh 的扩展模型更强大——可以替换一切——但学习曲线更陡（Cordis 上下文、服务注入、可逆效果、四种事件分发模式）。

#### 4.4 模型适配：内置生态 vs 配置行

| 维度                 | Pi                                                      | dsh                             |
|--------------------|---------------------------------------------------------|---------------------------------|
| Provider 数量        | 31 个内置 (v0.74.0)                                        | 任意 (通过插件)                       |
| 内置适配器              | llm-deepseek (DeepSeek 一等公民) + llm-pi-ai (通用 OpenAI 兼容) |                                 |
| 模型发现               | 自动发现可用模型                                                | 配置声明                            |
| 热切换                | <input type="checkbox"/> /model 或 Ctrl+L                | <input type="checkbox"/> 替换配置行  |
| 跨 Provider Session | <input type="checkbox"/> Session 可含多 Provider 消息        | <input type="checkbox"/> (模型无关) |
| 自定义 Provider       | 扩展实现                                                    | llm-pi-ai 可声明任意路由               |

深度分析：Pi 的 31 个 Provider 是开箱即用的优势。dsh 只有两个树内适配器 (llm-deepseek 和 llm-pi-ai)，但 llm-pi-ai 可以声明任意 OpenAI 兼容路由——“配置而非代码修改”。

#### 4.5 UI 形态：TUI vs Web

| 维度      | Pi                                             | dsh                                           |
|---------|------------------------------------------------|-----------------------------------------------|
| 主界面     | 终端 TUI (pi-tui，一等公民)                           | Web UI (dsh-web-app，一等公民)                     |
| 终端 UI   | <input type="checkbox"/> 差分渲染、组件系统、内联图片        | <input type="checkbox"/> 社区插件 (dsh-cc-tui 等)  |
| Web UI  | <input type="checkbox"/> (有 pi-web-ui 包但不是主界面) | <input type="checkbox"/> 工作台 (127.0.0.1:3080) |
| UI 可替换性 | 扩展可改编辑器、加 widget                               | UI 是 bundle，整个换                               |

深度分析：这是两个框架最显眼的差异。Pi 是终端原生的——TUI 是一等公民，差分渲染、组件系统、内联图片都有。dsh 是 Web 优先的——shipped profiles 是 web 和 headless，TUI 只是社区插件。终端原生开发者是 Claude Code 先赢得的人群，在 dsh 中明显排在第二位。

#### 4.6 安全与沙箱

| 维度   | Pi                                     | dsh                                                   |
|------|----------------------------------------|-------------------------------------------------------|
| 沙箱   | <input type="checkbox"/> 故意不做 (用容器)    | <input type="checkbox"/> 插件 (bwrap/Landlock/Seatbelt) |
| 权限系统 | <input type="checkbox"/> 故意不做 (用容器/扩展) | <input type="checkbox"/> 插件 (dsh-base 提供)             |



| 维度   | Pi                           | dsh                             |
|------|------------------------------|---------------------------------|
| 审批流程 | <input type="checkbox"/> 需扩展 | <input type="checkbox"/> 工具管道内置 |
| 凭据管理 | API key 在环境变量                | apiKeyEnv 引用模式 + Keychain       |

深度分析：Pi 的立场是"跑容器里"——安全是环境的事，不是 Agent 的事。dsh 内置了沙箱（Linux 用 bwrap，macOS 用 Seatbelt，支持 Landlock）和审批策略。对于企业部署，dsh 更完整；对于个人开发者跑在本地，Pi 更简单。

## 5. 记忆系统架构对比与启示

### 5.1 两个框架的记忆能力

| 维度       | Pi                                                          | dsh                                                                         |
|----------|-------------------------------------------------------------|-----------------------------------------------------------------------------|
| 事件钩子     | <input type="checkbox"/> 成熟 ( pi.on("tool_call") 等 20+ 钩子 ) | <input type="checkbox"/> <input type="checkbox"/> 不稳定 ( Developer Preview ) |
| 长期记忆 API | 扩展实现                                                        | 插件实现                                                                        |
| 状态持久化    | JSONL 文件                                                    | 事件流日志                                                                       |
| 上下文压缩    | <input type="checkbox"/> 内置 Compaction                      | 插件实现                                                                        |
| 记忆作用域    | project / session / global                                  | userId / agentId / runId                                                    |
| 异步记忆整合   | <input type="checkbox"/> 事件钩子可异步                            | <input type="checkbox"/> API 不稳定                                            |

### 5.2 Mem0 官方插件对比：同一团队，两种设计

这是最有说服力的对比——同一个 Mem0 团队，给两个平台各做了一个插件，设计完全不同：

| 维度       | Mem0 Pi 插件                              | Mem0 dsh 插件                                  |
|----------|-----------------------------------------|----------------------------------------------|
| 安装       | pi install<br>npm:@mem0/pi-agent-plugin | dsh plugin add +<br>cordis.example.yml       |
| 工具数      | 1<br>个 ( mem0_memory——搜索+存储合一 )         | 2 个 ( search_memory +<br>add_memory )        |
| 命令数      | 8 个斜杠命令                                 | 0 ( dsh 无斜杠命令概念 )                            |
| 技能数      | 8 个 SKILL.md                            | 0                                            |
| 自动捕获     | <input type="checkbox"/> 自动从对话中提取记忆     | <input type="checkbox"/> 计划中，等 dsh 事件 API 稳定 |
| Dream 整合 | <input type="checkbox"/> 合并重复、解决矛盾、清理过期 | <input type="checkbox"/>                     |
| 作用域      | project / session / global              | userId / agentId / runId                     |
| 记忆分类     | 10 个自动分类                                | 无 ( 依赖后端分类 )                                 |
| 后端       | Mem0 Cloud Platform                     | Mem0 Cloud Platform                          |
| 代码量      | ~10 个源文件 + 8 个技能                        | 6 个源文件，~500 行                                |

深度分析：差异原因不在 Mem0 团队的能力，而在两个平台的成熟度差异：

| 差异点      | 原因                                  |
|----------|-------------------------------------|
| 自动化程度    | Pi 有成熟的事件钩子；dsh 事件 API 不稳定          |
| 功能丰富度    | Pi 的扩展模型更成熟；dsh 是 Developer Preview |
| Dream 整合 | Pi 的事件钩子允许后台异步整合；dsh 的异步能力还在建       |
| UI 集成    | Pi 有 TUI 斜杠命令系统；dsh 没有              |

5.3 记忆注入的架构约束

在 dsh 中，记忆注入到模型上下文必须通过 `agent.inject()` 或 `agent/pre-step` 事件完成：

```
■■■■■
■■■■■ → agent/pre-step ■■ → enter(messages) → ■■■■ → ■■■■

■■■■■
■■■■■ → ■■■■■■■■■■ ← ■■■■■■ 2■■■■■ = ■■■■
```

在 Pi 中，记忆注入没有这样的架构约束——扩展理论上可以直接修改模型上下文（虽然不推荐）。但 Pi 的事件钩子（`pi.on("tool_call")` 等）提供了合规的注入路径。

5.4 对记忆模块开发的启示

| 如果在 Pi 上做                              | 如果在 dsh 上做                                                                    |
|----------------------------------------|-------------------------------------------------------------------------------|
| 事件钩子成熟，可以做全自动记忆                        | 事件 API 不稳定，先做手动工具，等稳定再加自动化                                                    |
| 用 <code>pi.on("tool_call")</code> 捕获对话 | 用 <code>agent/step</code> 事件捕获，但注意平台时序约束                                      |
| 可以做斜杠命令                                | 没有斜杠命令系统                                                                      |
| 可以做 SKILL.md 引导模型                      | 可以在 system prompt section 引导                                                  |
| <code>pi install</code> 一键安装           | <code>dsh plugin add + cordis.patch.yml</code>                                |
| 参考 @mem0/pi-agent-plugin 的全自动实现        | 参考 @mem0/deepseek-plugin 的最小实现 + <code>runfali/dsh-mem0-plugins</code> 的全自动实现 |

6. 架构成熟度与风险评估

6.1 成熟度对比

| 维度              | Pi                            | dsh                               |
|-----------------|-------------------------------|-----------------------------------|
| 版本              | v0.84.2 ( 活跃迭代 )              | v0.1.0-rc.5 ( Developer Preview ) |
| 稳定性承诺           | MIT，npm，JSONL on disk，JSON 配置 | MIT，承诺兼容性破坏变更                     |
| 发布节奏            | 频繁 ( 最新发布 2026-05-04 )        | 早期 ( 2026-08-13 开源 )              |
| 社区生态            | 扩展生态成熟                        | 插件生态数小时大                          |
| Claude Code 互操作 | 无                             | 原生支持 ( AGENTS.md、hooks、子 Agent )  |

| 维度   | Pi                 | dsh                            |
|------|--------------------|--------------------------------|
| 文档质量 | README + 官网 + 社区文档 | docs/architecture.md<br>架构文档优秀 |

## 6.2 Pi 的风险

| 风险             | 严重度 | 说明                                |
|----------------|-----|-----------------------------------|
| 核心不可替换         | 中   | 如果需要修改 Agent 基本行为模式，需要 fork       |
| 单人维护           | 中   | Mario Zechner 是主要维护者，bus factor 低 |
| OSS Weekend 模式 | 低   | 自动关闭非维护者的 issue/PR，可能影响社区贡献       |
| 版本漂移           | 低   | 频繁发布，latest 会移动                   |

## 6.3 dsh 的风险

| 风险                | 严重度 | 说明                                                               |
|-------------------|-----|------------------------------------------------------------------|
| Developer Preview | 高   | README 明确承诺兼容性破坏变更                                               |
| 事件 API 不稳定        | 高   | 记忆插件等依赖事件 API 的功能受影响                                             |
| 组合调试              | 中   | "哪一层设置了这个？"——需要<br>--dump-config diff                            |
| 学习曲线              | 中   | Cordis<br>上下文、服务注入、可逆效果、四种事件模式                                   |
| 模仿包风险             | 中   | 开源后数小时出现仿冒包，需认准官方 npm 包                                          |
| DeepSeek 模型限制     | 低   | DeepSeek 聊天路由仅文本，图像输入需其他 Provider                                |
| 自举悖论              | 低   | 仓库含 CLAUDE.md 和 .claude/——"DeepSeek 的 Harness 是用别人的 Harness 构建的" |

## 6.4 社区观察

"从 Claude Code 切换到 GPT 驱动的 agent，你很快发现：模型从来不是整个产品。你的权限重置了。你的记忆没了。你的命令不工作了。几个月的习惯一夜之间变得无用。"——这是 dsh 要解决的问题。

dsh 的"一切皆插件"直接回应了这个痛点——切换模型只是删除一个配置行，其他不动。Pi 通过 31 个 Provider 内置支持也部分解决了这个问题（/model 热切换）。

## 7. 对 Agent 框架设计范式的影响

## 7.1 三种设计哲学的本质

| 维度       | Pi                | dsh                    |
|----------|-------------------|------------------------|
| 核心策略     | 极简（故意不做）          | 不存在（一切皆插件）             |
| 设计原点     | "让用户自己加"          | "让用户替换一切"              |
| Agent 视角 | 终端编码助手            | 可重构的模型外壳               |
| 扩展方式     | TypeScript 扩展模块   | Cordis 插件 + YAML patch |
| UI 形态    | 终端 TUI（一等公民）      | Web UI（也是插件）           |
| Agent 循环 | 固定（核心代码）          | 可替换的插件                 |
| 模型适配器    | 内置支持 31+ provider | 一个插件行                  |
| 类比       | 精简改装车——底盘最小       | 乐高——连底盘都是积木            |

一句话总结：Pi 说"我不做这些功能，你来加"；dsh 说"我做的所有功能你都可以换掉"。

## 7.2 "一切皆插件"不是营销——是启动过程的字面描述

当 dsh 启动时，它组合一个配置行列表，每一行都来自一个插件。dsh --profile web --dump-config 打印整个运行系统，里面每一行都是某个 patch 可以定位的目标。这不是 debug 工具——这是架构的核心工作方式。

## 7.3 自修改 Agent 的可能性

Cordis 的时空可组合性买到了两个普通应用永远不给你的保证：

1. 时间可组合性：卸载插件完全撤销副作用 → Agent 可以在对话中途写一个插件并加载到自身
2. 空间可组合性：插件在兄弟变化时重新解析依赖 → 替换一个能力不需要 fork 整个系统

这意味着 Agent 可以自修改——不是在 prompt 层面，而是在架构层面。一个 Agent 可以在运行中卸载自己的工具、加载新工具、替换自己的循环策略，而所有这些都不需要重启。Pi 做不到这一点——核心循环不可替换。

## 7.4 会话日志作为唯一真相源的深层含义

dsh 的"模型可见 = 已记录"不变量的含义远超"有日志可查"：

- 可复现：任何模型请求都可以从日志精确重建
- 可分叉：从任意 turn 边界 fork 会话
- 可回放：重放事件流重现完整交互
- 可审计：逐字节验证外发请求
- 诚实 token 计费：token 计量从日志推导，没有隐藏消耗

对记忆模块的含义：记忆注入必须通过事件系统完成。你不能绕过日志直接修改模型上下文。你的记忆插件注入的内容必须成为会话事件，被记录、可回放、可审计。

## 7.5 凭据引用模式

两个独立代码库（dsh 和 Pi 生态中的 DirectorOS）都选择了"凭据是引用，不是值"——密钥在 Keychain/环境变量中，配置只持有指针。两个代码库独立到达同一结论，是该设计默认正确性的信号。

## 7.6 dsh 作为"元 Harness"

dsh 的子 Agent Seam 可以驱动 Claude Code、Codex、ACP 对等体——dsh 被设计为可以编排其他 Harness。这使它定位为"元 Harness"（meta-harness），而非简单的 Claude Code 替代品。Pi 不具备这种能力（不内置子 Agent）。

## 7.7 代价：组合调试

dsh 诚实地承认了代价：

"灵活性不是免费的。"

- 1. 每个插件都是可以移动的 API 表面
- 2. 调试是组合形状的——问题不是"这段代码做了什么"，而是"哪一层设置了这个"
- 3. 没有什么阻止你组合出不一致的东西

--dump-config vs --dump-default-config 的 diff 是回答"哪一层设置了这个"的工具。Pi 没有这个问题——核心是固定的，扩展的职责边界清晰。

# 8. 总结与选型建议

## 8.1 两个框架的本质差异

|         | Pi                | dsh                         |
|---------|-------------------|-----------------------------|
| 哲学      | 极简核心 + 激进扩展       | 无特权核心 + 一切皆插件               |
| 循环      | 固定 418 行，事件钩子干预   | 可替换插件，四种事件分发                |
| 状态      | JSONL 树（原地分支）     | 只追加事件流（运行时断言）               |
| 扩展      | TypeScript 模块（加法） | Cordis 插件（替换法）              |
| 卸载      | 可能留残余             | 完全撤销                        |
| UI      | TUI 一等公民          | Web UI 一等公民                 |
| 沙箱      | 不做（用容器）           | 内置（bwrap/Seatbelt/Landlock） |
| 子 Agent | 不做（用 tmux/扩展）     | 可驱动其他 Harness（元 Harness）    |
| 成熟度     | v0.84.2，活跃迭代      | Developer Preview，承诺破坏变更    |
| 记忆插件生态  | Mem0 全自动（8命令+8技能） | Mem0 纯手动（2工具），等 API 稳定      |

## 8.2 选 Pi 如果

- 工作在终端——Pi 的 TUI 是一等公民，体验远超 dsh 的 Web UI
- 喜欢极简哲学——不想要子 Agent、plan mode、权限弹窗等"多余"功能

- 要自己定制一切——TypeScript Extensions 能力极强
- 做编码任务——核心工具 ( read/write/edit/bash ) 就是为编码设计的
- 在本地跑——不需要 Web 服务器，终端直接跑
- 事件钩子成熟——记忆模块可以做全自动
- 用 DeepSeek 模型——Pi 原生支持 DeepSeek V4
- 要离线模式——Pi 有 --offline 和 /llama 命令
- 要可锁定版本——JSONL on disk，MIT，npm，扩展是 TypeScript

### 8.3 选 dsh 如果

- 要 Web UI——dsh 的 Web 工作台是核心体验
- 要替换 Agent 循环——这是 dsh 独有的能力
- 要沙箱/权限系统——dsh 内置 ( 通过 dsh-base )
- 要多平台网关——dsh 支持 webhook、Slack/Telegram 等
- 要子 Agent——dsh 可驱动其他 Harness ( 元 Harness 定位 )
- 要 MCP——dsh 内置支持
- 追求"无特权核心"理念——dsh 的 Cordis 架构是学术级的
- 从 Claude Code 迁移——dsh 有原生互操作 ( AGENTS.md、hooks、子 Agent )
- 用 DeepSeek 模型——dsh 是 DeepSeek 官方产品
- 追求自修改 Agent——Cordis 的时空可组合性使架构级自修改成为可能

### 8.4 对记忆模块研究的建议

| 如果在 Pi 上做                 | 如果在 dsh 上做                   |
|---------------------------|------------------------------|
| 事件钩子成熟，可以做全自动记忆提取         | 事件 API 不稳定，先做手动工具，等稳定再加      |
| 用 pi.on("tool_call") 捕获对话 | 用 agent/step 事件捕获，注意时序约束     |
| 参考 Mem0 Pi 插件的全自动实现       | 参考 Mem0 dsh 插件 + runfali 社区版 |
| 限制：核心循环不可替换               | 优势：可深度集成到循环中                 |
| 记忆注入无架构约束                 | 记忆注入必须通过事件系统 ( 架构约束 )        |

### 8.5 最终洞察

- Pi 和 dsh 不是竞品——它们是正交的。Pi 是"极简核心 + 激进扩展" ( 加法哲学 ) ；dsh 是"无特权核心 + 一切皆插件" ( 替换哲学 ) 。两者解决的是不同的问题。
- dsh 的架构领先于产品。Cordis 的时空可组合性、会话日志不变量、Seam 设计模式都是学术级的设计。但产品还是 Developer Preview，API 可能变。
- Pi 的成熟度领先于架构。v0.84.2，31 个 Provider，20+ 事件钩子，内置 Compaction，JSONL 树分支——这些都是实战验证过的。但核心循环不可替换。

4. 对于记忆模块研究：Pi 更适合现在做（API 稳定、事件钩子成熟）；dsh 更适合研究架构范式（不变量设计、可逆效果、Seam 模式）。
5. 两个框架都选择了"凭据是引用"模式——独立到达同一结论，是默认正确性信号。
6. dsh 的"模型可见 = 已记录"不变量值得任何 Agent 框架借鉴——它买到了确定性回放、诚实 token 计费 and 架构级审计。

## 附录 A：核心术语词典

| 术语     | 英文                           | 精确定义                                                       |
|--------|------------------------------|------------------------------------------------------------|
| 线束     | Harness                      | 套在模型外面的可重构外壳，提供工作空间、工具、权限和运行记忆                             |
| 内核     | Kernel (Cordis)              | 管理插件挂载/卸载/依赖的通用框架，非特定于 Agent                               |
| 插件     | Plugin                       | 导出 <code>apply(ctx)</code> 的模块，注册服务/事件/效果                  |
| 捆绑包    | Bundle                       | 声明了 <code>dsh.bundle</code> 清单的 npm 包，携带配置行和代码             |
| 配置文件   | Profile                      | <code>~/.dsh/profiles/</code> 下的命名组装体                      |
| 补丁     | Patch                        | YAML 配置行，按 id 定位，替换整行                                      |
| 缝接     | Seam                         | 可替换能力的抽象接口，有三个角色                                           |
| 会话日志   | Session Log                  | 只追加的事件流，模型上下文的唯一真相源                                        |
| 步骤     | Step                         | 一次模型请求 + 它调用的工具                                            |
| 轮次     | Turn                         | 零或多个步骤，从认领输入到不欠工作                                          |
| 时空可组合性 | Spatiotemporal Composability | Cordis 的学术贡献：卸载撤销副作用 + 兄弟变化时重新解析                           |
| 扩展     | Extension                    | Pi 的 TypeScript 模块，通过 <code>ExtensionAPI</code> 注册工具/命令/事件 |
| 技能     | Skill                        | SKILL.md 文件，渐进式披露，按需加载                                     |
| 会话压缩   | Compaction                   | Pi 内置的上下文压缩机制（自动+手动+分支摘要）                                  |

## 附录 B：关键参考资源

| 资源                     | 链接                                                                                                                                                              |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pi 仓库                  | <a href="https://github.com/badlogic/pi-mono">github.com/badlogic/pi-mono</a>                                                                                   |
| Pi 官网                  | <a href="https://pi.dev">pi.dev</a>                                                                                                                             |
| Pi 文档                  | <a href="https://pi.dev/docs/latest">pi.dev/docs/latest</a>                                                                                                     |
| Pi 架构概览（社区）            | <a href="https://github.com/mudrui/pi-mono-docs/blob/main/01-architecture-overview.md">github.com/mudrui/pi-mono-docs/blob/main/01-architecture-overview.md</a> |
| Pi DeepWiki            | <a href="https://deepwiki.com/badlogic/pi-mono">deepwiki.com/badlogic/pi-mono</a>                                                                               |
| Pi Agent Core DeepWiki | <a href="https://deepwiki.com/badlogic/pi-mono/3-pi-agent-core:-agent-framework">deepwiki.com/badlogic/pi-mono/3-pi-agent-core:-agent-framework</a>             |

| 资源                      | 链接                                                                                                                                                                                |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Armin Ronacher 的 Pi 分析  | <a href="https://lucumr.pocoo.org/2026/1/31/pi">lucumr.pocoo.org/2026/1/31/pi</a>                                                                                                 |
| MPisaac 的操作者级分析         | <a href="https://mpiv.ai/blog/pi-mono-deep-dive-the-minimalist-coding-agent-for-operators-2026">mpiv.ai/blog/pi-mono-deep-dive-the-minimalist-coding-agent-for-operators-2026</a> |
| DeepSeek API Pi 集成      | <a href="https://api-docs.deepseek.com/quick_start/agent_integrations/pi_mono">api-docs.deepseek.com/quick_start/agent_integrations/pi_mono</a>                                   |
| dsh 仓库                  | <a href="https://github.com/deepseek-ai/deepseek-harness">github.com/deepseek-ai/deepseek-harness</a>                                                                             |
| dsh 官网                  | <a href="https://deepseek.com/harness">deepseek.com/harness</a>                                                                                                                   |
| dsh 架构文档 ( 源 )          | <a href="https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/architecture.md">github.com/deepseek-ai/deepseek-harness/blob/master/docs/architecture.md</a>           |
| dsh DeepWiki            | <a href="https://deepwiki.com/deepseek-ai/deepseek-harness/2-core-architecture">deepwiki.com/deepseek-ai/deepseek-harness/2-core-architecture</a>                                 |
| Dwarves Memo 深度分析       | <a href="https://memo.d.foundation/deepseek-harness-architecture">memo.d.foundation/deepseek-harness-architecture</a>                                                             |
| rustman 分析              | <a href="https://rustman.org/wiki/deepseek-harness">rustman.org/wiki/deepseek-harness</a>                                                                                         |
| dsh-in-depth            | <a href="https://dsh-in-depth.com/architecture/overview">dsh-in-depth.com/architecture/overview</a>                                                                               |
| Inside DeepSeek Harness | <a href="https://renatomignone.github.io/inside-deepseek-harness">renatomignone.github.io/inside-deepseek-harness</a>                                                             |
| Cordis 论文               | <a href="https://arxiv.org/abs/2608.25512">arxiv.org/abs/2608.25512</a>                                                                                                           |
| Cordis 仓库               | <a href="https://github.com/cordiverse/cordis">github.com/cordiverse/cordis</a>                                                                                                   |
| Mem0 Pi 插件              | <a href="https://github.com/mem0ai/mem0/tree/main/integrations/pi-agent-plugin">github.com/mem0ai/mem0/tree/main/integrations/pi-agent-plugin</a>                                 |
| Mem0 dsh 插件             | <a href="https://github.com/mem0ai/mem0/tree/main/integrations/deepseek-plugin">github.com/mem0ai/mem0/tree/main/integrations/deepseek-plugin</a>                                 |
| DSHKit 社区指南             | <a href="https://dshkit.dev/plugins/everything-is-a-plugin">dshkit.dev/plugins/everything-is-a-plugin</a>                                                                         |

文档版本：v1.0

核心结论：Pi-Agent 和 DeepSeek Harness 代表了两种正交的 Agent 框架设计哲学——Pi 是"极简核心 + 激进扩展"（故意不做的功能让用户自己加，418 行循环 + 20+ 事件钩子），dsh 是"无特权核心 + 一切皆插件"（做的所有功能你都可以换掉，空列表启动 + Cordis 时空可组合性 + 会话日志不变量）。dsh 的架构设计领先（学术级），但产品成熟度落后（Developer Preview）；Pi 的产品成熟度领先（v0.84.2），但架构灵活性受限（核心循环不可替换）。对于记忆模块研究，Pi 更适合现在做（API 稳定），dsh 更适合研究架构范式（不变量设计、可逆效果、Seam 模式）。两个框架独立到达的相同结论（凭据引用模式、模型可见即记录）值得任何 Agent 框架借鉴。